

raffle.fun at a Glance

READER

Participants, sponsors, partners, press

PROTOCOL COMMIT

5772e54ba89c

NOT DEPLOYED

NOT INDEPENDENTLY AUDITED

Pre-release

raffle.fun is an NFT raffle protocol written as immutable smart contracts. A sponsor locks one NFT into a purpose-built contract; anyone buys numbered tickets in a stablecoin; one ticket is drawn using Pyth Entropy; the holder of that ticket claims the prize. Every rule is fixed when the raffle is created, and nobody — including the people who wrote it — can change it afterwards.

What a sponsor does

The sponsor calls `createRaffle`, setting the prize, ticket price, minimum ticket threshold, and sale window. In that single transaction the protocol deploys a new `Raffle`, registers it, and pulls the NFT into escrow, verifying it actually arrived. If any step fails the whole transaction reverts — so a raffle can never exist without its prize already locked inside it.

What a buyer receives

Tickets are ERC-721 NFTs minted by the raffle itself, numbered sequentially from 1, at a flat price with no checkout fee. **Ticket ownership is the bearer claim credential.** Whoever holds the ticket at redemption time can claim — the protocol never consults purchase history, and an approved operator is not enough. Tickets trade freely during the sale, which also means you can lose the claim by sending one somewhere that cannot call back into the contract.

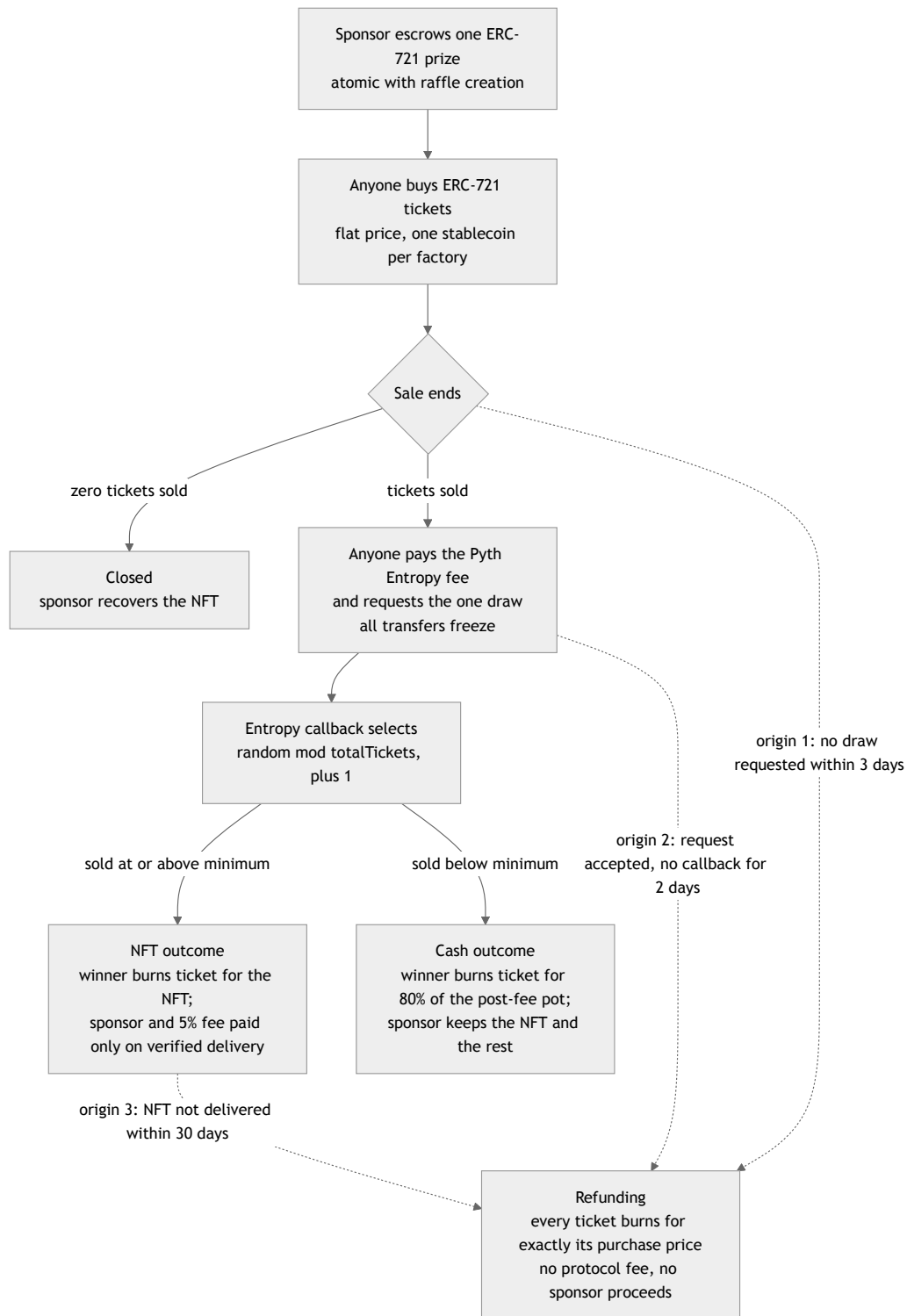
How a winner is selected

After the sale closes, anyone may pay Pyth Entropy's current fee and request the one draw, within a three-day window. The moment that request is submitted, **all ticket transfers freeze**, fixing ownership before the randomness provider can know the outcome. The Entropy callback then selects ticket `(random mod totalTickets) + 1`.

The two success outcomes

If at least `minimumTickets` sold, the drawn ticket wins **the NFT**. The pot stays escrowed until the winner burns their ticket and delivery is verified onchain; only then are the sponsor and the fee paid. If the threshold was missed, the drawn ticket wins **cash** — 80% of the post-fee pot — and the sponsor keeps the NFT plus the rest.

The three refund fallbacks



All three origins are finalized by the same permissionless function, `enableRefunds()`, which anyone may call once the relevant deadline passes. Each outstanding ticket burns for exactly one ticket price, in batches of up to 100. A raffle that sold zero tickets is closed instead, returning the NFT to the sponsor's recovery address.

A deadline does not change state by itself: at each boundary a valid callback and a refund finalization are both executable, and whichever lands first wins.

Fee and control

The protocol charges **5% of gross sales**, floored, and only when a raffle actually resolves. Every refund path charges nothing. The rate is a compile-time constant baked into every deployed raffle and cannot be changed for an existing raffle by anyone.

An existing `Raffle` has **no administrator** — no owner, pause, upgrade, rescue, or settlement override. The factory owner controls two future-facing things: which treasury *new* raffles pay, and whether *new* raffles can be created. Neither touches an existing raffle. Factory ownership cannot be renounced.

Key risks

- **Pyth Entropy provider trust.** The provider can know the result before revealing it. Transfer locking prevents buying the winning ticket after the draw, but a provider that already holds tickets could publish only favorable results and take a refund otherwise. Modelled, its advantage peaks at a quarter of the pot when it holds half the tickets. This is a documented, **unresolved** High-severity trust assumption.
- **USDC issuer controls.** Circle can pause, freeze, blacklist, or upgrade the token. No contract can force a frozen transfer.
- **Base liveness and censorship.** The sequencer can delay or censor a draw request or callback into a refund. A halted chain has no application-level escape.
- **Malicious or upgradeable NFTs.** A prize collection can lie about its own standard and ownership, or be paused or burned after escrow. Buyers bear all prize due diligence.
- **Smart-contract risk.** Never independently audited, never run in production.
- **Unsafe ticket destinations.** A ticket sent to a lost key or a contract that cannot redeem forfeits the claim permanently. There is no admin recovery.
- **Legal and gaming regulation.** Raffles and prize promotions are regulated differently in every jurisdiction. No legal review has been performed.
- **Worthless leftovers.** Losing tickets are not burned. They stay in wallets and stay tradable after a raffle settles, so a secondary-market buyer can be sold a settled ticket that looks identical to a live one.
- **No privacy.** Sponsors, purchases, ticket ownership, the winner, and all amounts are public.

Status

ITEM	STATUS
Protocol status	Pre-release. Feature-complete contracts; the release checklist states verbatim "not release-ready".
Deployment status	Not deployed. No deployment record exists for any public network; the repository contains no live address, and the web app disables writes without one.
Internal review status	An internal adversarial hardening campaign has been completed, including fuzzing, stateful and strict invariants, differential modelling, Echidna and Medusa, Halmos symbolic checks, mutation testing, Slither, and pinned Base fork validation. The maintainers state this is evidence, not proof, and not an audit.
Independent audit status	None. No external audit has been performed. The most recent review is self-administered and describes itself as "not an independent audit or a production authorization".

ITEM

STATUS

Source commit `5772e54ba89c06646815ed52a881cd8940f094ca`

raffle.fun must not be described as audited, trustless, provably fair, guaranteed, live, or production-ready. Every claim on this page traces to [docs/facts/raffle-fun-facts.md](#), which cites the exact contract, function, and test behind it.